☕ Spring Framework

Aspect Oriented Programming (AOP)

Complete Study Notes

How to Achieve AOP in Spring

Spring provides two major ways to achieve Aspect Oriented Programming:

A) Spring's Built-in AOP Framework

Spring provides its own AOP framework that simplifies the implementation of AOP.
It offers declarative transaction management and lightweight proxy-based AOP.

Spring's built-in AOP Framework provides 2 ways to achieve AOP:

1 Spring XML Configuration

- **1.1 DTD Based (Older Approach)**
  - Uses Document Type Definition for bean configuration
  - Less strict syntax, limited features
- **1.2 XSD Based (Modern Approach)** ✅
  - Uses XML Schema Definition — more powerful and strict
  - Supports data typing, namespaces, element grouping

2 Java & Annotation-Based Configuration

- **Uses @Configuration and @EnableAspectJAutoProxy**
  - Also known as Programmatic Configuration
  - Most commonly used in modern Spring applications

⚡ B) Using AspectJ Framework in Spring

The AspectJ framework provides more advanced AOP capabilities which can be integrated with Spring applications.
More powerful than Spring's built-in AOP — supports compile-time, post-compile, and load-time weaving.

We can achieve AOP using AspectJ in 2 ways:

1 XML Configuration (Advanced)

- Verbose but clear and explicit configuration
- Uses <aop:config> tags in Spring XML

2 Annotations (Concise & Widely Used)

- Uses @Aspect, @Before, @After, @Around, @Pointcut annotations
- Concise, readable, and the most popular approach today

Quick Summary

A) Spring Built-in AOP → XML (DTD / XSD) or Java/Annotation (@EnableAspectJAutoProxy)
B) AspectJ Framework → XML (Advanced) or Annotations (@Aspect, @Before, etc.)

Difference Between DTD and XSD

Both DTD and XSD are used to define the structure and rules of XML documents, but they differ significantly in capabilities.

DTD (Document Type Definition)	XSD (XML Schema Definition)
Document Type Definition	XML Schema Definition
Older approach — legacy technology	✅ New, modern approach
Less extensible — lacks data typing & element grouping	Highly extensible — supports data typing, element grouping, etc.
Limited namespace support	Highly robust namespace support
Simpler and less strict syntax	More complex and strict syntax — reduces errors
No data type validation (everything is text)	Strong data type validation (integer, date, boolean, etc.)
Cannot inherit or extend definitions	Supports inheritance and extension of types

---

Spring XML Configuration — DTD Based

This is an older approach where we define aspects, advices, pointcuts etc. using XML-based configuration.
DTD-based configuration is not recommended for new projects. Prefer XSD or annotation-based config.

Key Characteristics:

- Uses DOCTYPE declaration at the top of XML config file
- Defines beans using older spring-beans.dtd schema
- Less flexible — no data type enforcement
- Replaced by XSD-based config in Spring 2.0+

Example (DTD-based XML header):

```
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
   "http://www.springframework.org/dtd/spring-beans-2.0.dtd">
<beans>
   <bean id="myBean" class="com.example.MyClass"/>
</beans>
```